

Vision-Language-Action Franka Manipulation MuJoCo to Isaac Lab Sim2Sim Transfer

Detailed execution plan for a 2-person team

Project goal: Build a language-guided tabletop manipulation system for a Franka Panda. A simulated RGB camera provides scene images. The policy receives the image, natural-language instruction and robot state and predicts Cartesian end-effector commands. A Cartesian/IK controller executes the commands. Develop and train in MuJoCo, then evaluate the same policy in Isaac Lab.

1. Final system

```
Simulated RGB camera -----> RGB image
Natural-language instruction ----+
Robot/end-effector state -----+
|
| v
| Learned VLA policy
|
| v
| [dx, dy, dz, dyaw, gripper]
|
| v
| Cartesian target / IK
|
| +-----+-----+
| v v
| MuJoCo Franka Isaac Franka
```

Use one fixed third-person RGB camera initially. Keep the gripper pointing downward by fixing roll and pitch. Use yaw only when object orientation matters. The main learned action is a small Cartesian movement plus a gripper command.

2. Tabletop task

Component	Plan
Robot	Franka Panda
Scene	Table, 3-5 objects, 2-3 bins/target zones, optional distractors
Vision	One simulated third-person RGB camera
Language	Pick/place commands, paraphrases and spatial instructions
Main action	[dx, dy, dz, dyaw, gripper]
Training simulator	MuJoCo
Transfer simulator	Isaac Lab / Isaac Sim
Learning	Behavior cloning from scripted expert demonstrations

Example tasks

Level	Instruction
Basic	Put the red cube in the blue bin.
Basic	Place the green block in the left bin.
Paraphrase	Move the crimson block into the blue container.
Spatial	Move the object left of the green cube into the blue bin.
Spatial	Put the object closest to the red cube in the left bin.

Level	Instruction
Clutter	Pick the red cube while ignoring the other objects.

3. Team ownership

Person 1 - VLA / Machine Learning	Person 2 - Robotics / Simulation
PyTorch project and training pipeline	MuJoCo Franka environment
Image/language/state preprocessing	RGB camera setup
Instruction generator	Cartesian controller
Dataset loader and normalization	Inverse kinematics
Semantic VLA baseline	Scripted expert pick/place
Continuous Cartesian behavior cloning	Expert trajectory generation
Language and visual generalization	Workspace limits and safety checks
Domain-randomized model training	Isaac Lab Franka environment
Model evaluation and ablations	MuJoCo/Isaac common adapter
Optional OpenVLA integration	Sim2Sim execution and controller analysis

Shared: repository structure, coordinate conventions, action definitions, integration tests, experiment design, final evaluation, report and demo.

4. Repository structure

```
project/
configs/
environments/
mujoco_env/
isaac_env/
controllers/
cartesian_controller.py
ik.py
data/
generation/
datasets/
models/
semantic_baseline/
cartesian_policy/
training/
evaluation/
scripts/
results/
demos/
README.md
```

5. Week-by-week execution

Week 1 - Setup and foundations

Person 1: Set up Python/PyTorch environment. Implement a small supervised-learning test. Learn tensors, Dataset/DataLoader, training loop, validation and checkpoint saving.

Person 2: Install MuJoCo. Load Franka Panda. Inspect arm joints, gripper, end-effector frame, actuators and camera rendering.

Together: Create Git repository. Define meters/radians, world frame, end-effector frame, gripper encoding and action format.

Exit: Franka scene runs and Person 1 has a working ML training pipeline.

Week 2 - Build the MuJoCo tabletop environment

Person 2: Add table, colored cubes/objects, bins/target zones and one RGB camera. Implement reset(seed), random object placement, RGB rendering and state queries.

Person 1: Create instruction templates and scene metadata format. Generate instructions from scene/task definitions.

Exit: Generate hundreds of randomized scenes with corresponding RGB images and valid language commands.

Week 3 - Cartesian Franka controller

Person 2: Implement end-effector target control and IK. Implement gripper open/close. Build scripted phases: approach object, descend, close gripper, lift, move above target, descend, open and retreat. Add workspace limits.

Person 1: Implement episode logging and automatic task-success checks.

Exit: Scripted controller succeeds reliably across randomized object positions.

Week 4 - Semantic VLA baseline

Person 1: Build an RGB + language model using pretrained image/text features and a small trainable network. Predict selected object and destination. Train and evaluate on randomized scenes.

Person 2: Connect predicted object/target to the scripted controller.

Exit: Natural-language command + camera image produces a complete pick/place episode in MuJoCo.

Week 5 - Generate expert trajectories

Person 2: Modify the scripted controller to output a small expert Cartesian action at every control step. Automatically run many randomized episodes.

Person 1: Define trajectory storage, train/validation/test splits and dataset validation scripts.

Store each step: RGB image, instruction, end-effector position, yaw, gripper state, expert $dx/dy/dz/dyaw$ /gripper, episode ID and task metadata.

Exit: Clean behavior-cloning dataset with thousands of control steps.

Week 6 - Train continuous Cartesian policy

Person 1: Build model: image encoder + text encoder + robot-state encoder -> fusion network -> Cartesian action head and gripper head. Normalize continuous actions. Train using behavior cloning.

Person 2: Build action clipping, target integration and a stable interface for executing predicted actions.

Exit: Model predicts held-out expert actions with reasonable error.

Week 7 - Closed-loop learned manipulation

Together: Run: capture RGB -> read robot state -> policy prediction -> execute Cartesian action -> simulation step -> new observation. Repeat until success, failure or timeout.

Person 1: Tune training and analyze model errors.

Person 2: Diagnose IK, grasp and controller failures.

Exit: Learned policy completes full pick/place tasks in MuJoCo.

Week 8 - Generalization experiments

Person 1: Add held-out paraphrases and spatial-language commands. Run full model, text-only and image-only ablations.

Person 2: Add unseen object layouts and clutter/distractors.

Evaluate: seen layouts, unseen positions, paraphrases, spatial relations and clutter.

Exit: Generalization results table and failure examples.

Week 9 - Build Isaac Lab version

Person 2: Recreate Franka, table, objects, bins and camera in Isaac Lab. Match workspace, action units, gripper convention and task success conditions. Implement the same simulator API.

Person 1: Make preprocessing independent of simulator-specific code.

Together: Run the scripted controller in Isaac first.

Exit: Same task can be executed through the common interface in both simulators.

Week 10 - Zero-shot Sim2Sim evaluation

Together: Freeze the MuJoCo-trained learned policy. Run it directly in Isaac Lab without retraining.

Person 1: Measure perception/model/action errors.

Person 2: Measure controller, camera, coordinate and physics differences.

Exit: MuJoCo vs Isaac success rates and a categorized list of transfer failures.

Week 11 - Domain randomization

Person 2: Randomize MuJoCo camera pose slightly, lighting, table/background appearance, object positions, scale and color shades while preserving task semantics.

Person 1: Regenerate/reuse demonstrations as appropriate and train the randomized model.

Together: Repeat zero-shot Isaac evaluation.

Exit: Quantitative comparison of normal training vs domain-randomized training.

Week 12 - Final evaluation and demo

Person 1: Produce final metric tables, plots, ablations and ML failure analysis.

Person 2: Produce controller/Sim2Sim analysis, reproducible simulator scripts and demo recordings.

Together: Run a fixed final test set and record matched MuJoCo/Isaac episodes. Clean repository and README. Prepare presentation/report.

Exit: Interview-ready project with code, results, demo and technical explanation.

6. Dataset specification

Each control-step sample should contain:

Field	Content
rgb	Camera RGB frame
instruction	Natural-language task
ee_position	Current [x, y, z]
ee_yaw	Current end-effector yaw
gripper_state	Current gripper state
action_xyz	Expert [dx, dy, dz]
action_yaw	Expert dyaw
action_gripper	Open/close or continuous gripper command
episode_id / timestep	Trajectory indexing
task metadata	Object/target identity and poses; randomization parameters

Split by **episodes/scenes**, not individual frames, so frames from one trajectory do not leak across training and test sets.

7. Main model

```
RGB image -----> image encoder -----+
|
Instruction -----> text encoder -----+--> feature fusion --> MLP/action head
| |
Robot state -----> state encoder -----+ +--> dx, dy, dz, dyaw
+--> gripper
```

Start with pretrained image/text encoders and a small trainable fusion/action network. Train the continuous action output with a regression loss and the gripper output with an appropriate classification or regression loss.

8. Simulator interface

```
reset(seed)
get_rgb()
get_robot_state()
get_end_effector_pose()
apply_cartesian_action(action)
step()
is_success()
is_failure()
```

Implement the same interface for MuJoCo and Isaac Lab. Keep model code outside both simulator implementations.

9. Required experiments

ID	Model / setting	Train	Test
E1	Scripted expert	N/A	MuJoCo
E2	Semantic VLA baseline	MuJoCo data	MuJoCo
E3	Learned Cartesian policy	MuJoCo demos	MuJoCo
E4	Learned Cartesian policy	MuJoCo demos	Isaac zero-shot
E5	Cartesian policy + domain randomization	Randomized MuJoCo demos	MuJoCo
E6	Cartesian policy + domain randomization	Randomized MuJoCo demos	Isaac zero-shot
E7	Full model / text-only / image-only	MuJoCo demos	Held-out MuJoCo tasks

10. Evaluation metrics

Metric	Report
Task success rate	Successful complete tasks / attempted tasks
Grasp success	Successful grasps / grasp attempts
Placement success	Correct placements / placement attempts
Action error	Mean error against expert actions on held-out data
Steps to completion	Average control steps for successful episodes
Language generalization	Success on held-out paraphrases
Spatial generalization	Success on relational/spatial instructions
Unseen-layout success	Success on held-out object positions
Sim2Sim transfer	Isaac success using MuJoCo-trained weights
Sim2Sim gap	MuJoCo success minus Isaac success
DR improvement	Isaac success after domain randomization minus baseline

11. Failure logging

Category	Examples
Vision / grounding	Wrong object or target selected
Language	Instruction or spatial relation misunderstood
Action prediction	Movement goes in wrong direction / oscillates
IK / controller	Target unreachable, unstable execution
Grasp	Bad approach, missed grasp, dropped object
Placement	Wrong location or release timing
Sim2Sim visual gap	Policy reacts differently to Isaac rendering
Sim2Sim dynamics gap	Correct-looking command executes differently

12. Domain-randomization implementation

Parameter	Implementation
Object XY positions	Random valid positions with minimum separation
Camera	Small translation/rotation/FOV jitter
Lighting	Vary intensity and direction
Table/background	Several neutral materials/appearances
Object colors	Vary shades while retaining semantic color labels
Object size	Small scale variation
Distractors	Random extra objects in later training

13. Optional extensions after the core project works

Extension	Work
Absolute Cartesian policy	Train [x,y,z,yaw,gripper] and compare with delta actions
Full orientation	Add roll/pitch or a suitable orientation representation
Action chunking	Predict multiple future Cartesian actions per inference
OpenVLA	Adapt an established VLA as an additional model baseline
More cameras	Add wrist or second external camera
More objects/tasks	Shapes, stacking, multiple sequential placements

14. Milestone gates

Gate	Must work before continuing
G1	MuJoCo Franka + camera + randomized tabletop scene
G2	Reliable scripted Cartesian pick/place
G3	Validated expert trajectory dataset
G4	Learned policy produces stable closed-loop MuJoCo behavior
G5	Isaac task works with scripted controller
G6	Frozen MuJoCo-trained policy tested in Isaac
G7	Domain-randomization experiment completed

15. Final deliverables

1. MuJoCo Franka environment with RGB camera and randomized tasks.
2. Cartesian/IK controller and scripted pick/place expert.
3. Automatic expert demonstration generator.
4. Semantic vision-language baseline.
5. Continuous Cartesian behavior-cloning policy.
6. Closed-loop MuJoCo evaluation suite.
7. Language, spatial and visual generalization tests.
8. Isaac Lab equivalent environment and common adapter.
9. Zero-shot MuJoCo-to-Isaac Sim2Sim results.
10. Domain-randomization experiment.
11. Metrics, plots and failure taxonomy.
12. Clean repository with installation and reproduction instructions.
13. Final report/presentation.
14. Demo video showing matched MuJoCo and Isaac tasks.

16. Final demo sequence

Demo 1: Show randomized MuJoCo scene and RGB camera view.

Demo 2: Run a basic language command with the learned policy.

Demo 3: Run a paraphrased or spatial command.

Demo 4: Show the same task in Isaac Lab using the MuJoCo-trained policy.

Demo 5: Show baseline Sim2Sim result vs domain-randomized model.

Demo 6: Present final success-rate table and representative failure cases.